Lab Assignments
Computer Graphics DT519A,
Örebro University, fall 2023

December 5, 2023

4 Ray based rendering

In this fourth lab we will move away from projective rendering, and instead work with ray-based computer graphics. However, we will still implement it in GLSL, to make use of the massive parallelisation that the GPU allows. We will base these tasks on the vertex and fragment shaders that you already have experience with. (On modern GPUs there are also other alternatives for implementing real-time ray tracing, including general-purpose compute shaders, and the recent dedicated ray-tracing shaders present in the DXR and Vulkan frameworks.)

Mandatory

4.1 Ray casting

Introduction

This section gives an introduction to the ray tracing framework set up in the code skeleton. For your actual tasks, see page 3.

Ray casting is the simplest form of ray-based rendering. It amounts to the following steps, for each pixel:

1. Define a ray going through the pixel, starting at the camera position $\vec{o}$.
2. Find the first object intersected by the ray.
3. Compute colour at the ray/object intersection and set pixel color.

The essential things we need to make this happen are thus a camera, a scene, and rays (lots of them)!

The Camera To fully determine the required parameters for the camera (see Figure 1), the following needs to be specified.

Image size Width and height, in pixels, of the image plane (shown in gray in Figure 1). This is modified when the window is resized.

Focal distance Distance between the camera $\vec{o}$ and the image plane (in pixels, too!). Modifying this distance has the effect of zooming in/out.

View transform As in the previous labs, we need the transformation from world to camera coordinates. Breaking it down step by step, we need the following.

Camera origin The translation offset from the world's origin to the camera's current position. This is modified by moving i.e. walking, lifting or lowering the camera. In the previous labs, this was the only parameter that was modified, but in this lab we'll also rotate the camera. This is `i_position` in Figure 1.

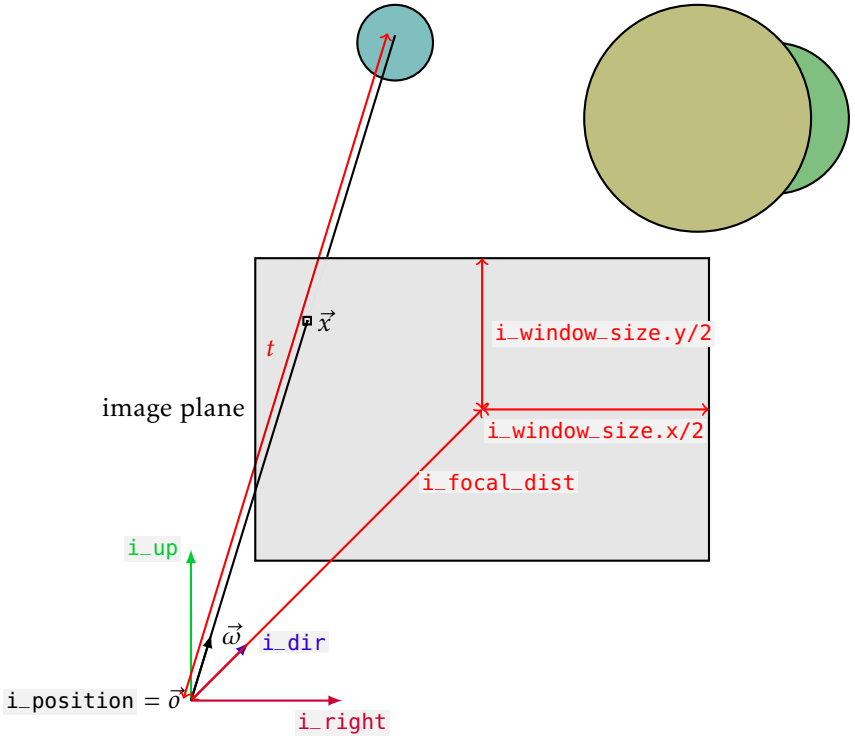


Figure 1: Schematic layout of the camera. The blue axis is the view direction, the other two axes form an orthogonal basis aligned with the pixel coordinates. The image plane is shown in gray.

Viewing direction The view vector `i_dir` shown in blue in Figure 1. This is modified by pitch and yaw angles.

Right direction The vector `i_right` shown in red in Figure 1. This is modified by yaw and roll angles.

Up direction The vector `i_up` shown in green in Figure 1. This is modified by roll and pitch angles (but can be found as the cross product of the view and right directions, if computed in that order).

The skeleton code already implements functionality for translating the camera with the arrow keys on the keyboard, and rotating it with the mouse. The camera parameters (`i_position`, `i_dir`, etc) are passed from the CPU side to the shader with `uniform`s.

A ray A ray R is simply a point of origin, a directional unit vector and a scalar term: $R = \vec{p}_0 + t\vec{\omega}$, with $t \geq 0$. As both the origin $\vec{p}_0$ and the direction $\vec{\omega}$ are in $\mathbb{R}^3$, so is R. To obtain rays we simply set $\vec{p}_0$ to the camera position $\vec{o}$, and get $\vec{\omega}$ from $\vec{x} - \vec{o}$, where $\vec{x}$ is the 3D coordinate of the current pixel. If we normalize this vector, we get $\vec{\omega}$. For any value of t we then obtain a 3-dimensional point along that ray. The trick is then to find the value for t that intersects the nearest surface in that direction.

In the fragment shader skeleton code, `struct Ray { vec3 origin; vec3 dir; }` defines a simple ray structure. The scalar t that gives the ray its length is kept elsewhere as each ray will have several possible t values, for each object that it intersects in the scene.

The scene The scene defines the geometry and also provides descriptions of the material properties and lights in the environment.

In our naive implementation, each ray will be tested against every object, potentially producing several intersections per ray, the closest of which has to be found. Most of the work in optimising rendering systems takes place here, by reducing the number of objects that have to be tested against each ray.

The skeleton code defines the scene in the `init()` function, where a few spheres, a plane and a light source are placed in the world.

Intersections The most interesting things in a ray caster/tracer happen when rays intersect objects.

Given a scene and a ray, we will need to compute their intersection. At the intersection, we need all the information required to compute the colour at that point; i e, the point itself, the surface normal at the point, and the parameters of the material used to define a BSDF (such as albedo, specularity, transparency).

There is a `struct Intersection` defined in the fragment shader, which should contain the information necessary from a successful ray–object intersection: that is, the point of intersection, the surface normal at that point, as well as the material properties of the object.

In a C++ implementation, the material should be an object, to allow for more flexibility. In this case, we have a `struct Material` that defines a set of parameters that can be used for any of a number of BSDF material models. In addition to the surface parameters you have used in previous labs, this code also includes an *emissive* colour (for objects that emit light), and transparency.

The function `Intersection intersect(Ray)` tests for intersection with all the objects in the scene, and assigns material properties to the `Intersection` based on which object was hit. For the plane, it additionally creates a (hard-coded) procedural chess-board texture.

Tasks

1. On the CPU side, specify the vertices for two triangles forming a rectangle in the XY plane with corners at $[-1,-1,0]$ and $[1,1,0]$ and draw them to cover the entire screen (similar to what you did in Task 1.2). As far as the CPU and the vertex shader stage are concerned, the scene only consists of these two triangles. All the magic will happen when computing the colours in the fragment shader.

   The given fragment shader code does a lot of things, as you can see in `lab4-1_fs.glsl`! However, the last thing it does in the main loop is to set the output colour to a fixed value. Leave it like that for the time being.

   Now you should see something that looks like Figure 2a.

   (Note that there are several uniforms in the C++ file that define the camera parameters, and that there is corresponding mouse callback code to update the camera view direction etc.)

2. Ignore the vertex shader :-) It will just pass on the vertex positions without any transformations. All the transformations that are needed will now be computed in the fragment shader instead!

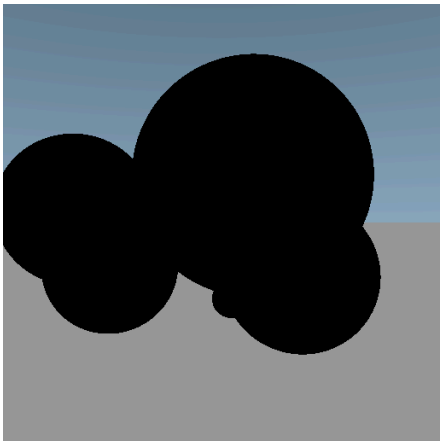
3. In the fragment shader, remove the last line in the main loop, which overwrites `frag_colour`.

   Now you have to shoot a ray for each call to the fragment shader, through the current pixel, as shown in Figure 1. In the `main` of the fragment

(a) Two triangles spanning the whole window. Not too exciting so far...



(b) Ray casting without objects, just showing a background sky sphere.



(c) Ray casting with spheres and a plane, but no materials.

Figure 2: Setting up ray casting.

shader, generate a ray for the current pixel, according to the camera parameters (`i_dir` etc).

Use `gl_FragCoord` in the fragment shader to get the xy coordinates in the image plane of the pixel that this fragment shader is processing. Since you also have `i_window_size`, `i_focal_dist`, etc, you can compute the direction of the ray and pass it to the `raycast` function. Just shoot a ray from wherever `i_position` happens to be through the image plane defined by the uniforms, and off you go. That is, create a variable of type `Ray` with the correct position and direction. To get the ray direction, add together `i_dir`, `i_up` and `i_right`, scaled by the focal distance and the xy fragment coordinates, respectively.

Now you should instead see a sky sphere as in Figure 2b, and be able to move the camera around with your mouse and keyboard. If not, your ray computations are not correct.

Please understand that as far as the normal rendering pipeline is concerned, you are still just rendering two triangles at fixed clip-space coordinates. The vertex shader does not apply any changed view matrix.

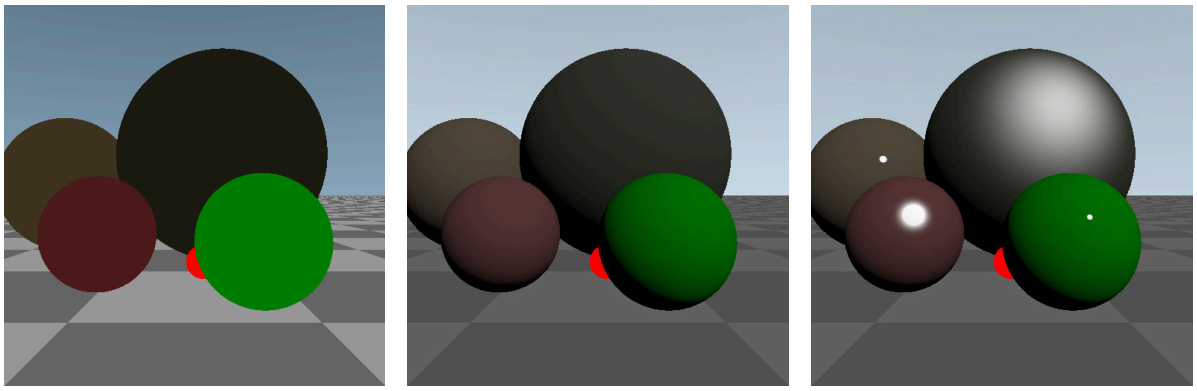


Figure 3: The scene from Task 4.1, rendered with ray casting. *Left:* Flat shading. *Centre:* Lambertian shading. *Right:* Blinn–Phong.

The corresponding code is instead running in the fragment shader, to compute changing colours on your two fixed triangles. (In fact, there are no model/view/projection matrices as such now, since we are not doing projective rendering.)

4. Complete the code for the intersection tests for the spheres and plane in the two `float intersect` functions, returning a distance t. See information in the lecture slides and in the course literature.

   If the intersection tests are correct, the result should look like Figure 2c.

5. Complete the rest of the function `Intersection intersect(Ray)`, populating `I` with the proper values, if the ray has hit a sphere or a plane. That is, set the point of the hit, and the normal and the material at that point.

6. Finally, compute the color of the pixel. This should be done at the end of the `raycast function`, when you have the surface parameters at the ray intersection. The supplied code sets a fixed plain colour (`isect.material.color_diffuse + isect.material.color_emission`).

   You should implement shading with a BRDF from Lab 2 here. For example, Lambertian shading. *Optionally*, you can re-implement Blinn–Phong or Cook–Torrance from Lab 2.

   One of the spheres in the scene emits red light. After computing the BRDF (and scaling it with the cosine of the incident light angle), you should also add the *emissive* colour of the object.

   When computing the colour, it is important to make sure that you use expressions like $\max(0, \cos(\alpha))$ for angles between the light and the normal. If you don't do that, you may see that the parts of objects that are not facing the light are pitch black, disregarding the emissive light (since you will be adding negative terms where $\cos(\alpha) < 0$).

The result should look similar to Figure 3. The result is visually similar to regular projective rendering. The difference is only in how we determine what is to be drawn on the image. Instead of each object generating a set of candidate pixels that have to be sorted in image space, each pixel will result in a list of candidate objects in 3D space.

Questions

Question 1 How does your image change if you increase or decrease `i_focal_dist`? Can you explain this?

Question 2 How could you implement super-sampling for anti-aliasing here, to get smoother edges of the spheres, and less aliasing issues on the checkerboard texture?

Mandatory

4.2 Ray tracing

Raytracing takes a similar approach to raycasting, but adds additional rays to deal with transparency, reflection and cast shadows. Your task is to upgrade the raycaster to include additional rays that are produced in 3 different cases:

1. For all surfaces: a secondary ray ("shadow feeler") is cast from the surface in the direction of the light source. If this ray intersects any object on its way towards the light, this surface is in shadow (and should recieve only ambient light). If not, the colour is computed in the same way as in task 4.1.

2. For surfaces with a mirror component: a secondary ray is cast from the surface, taking the incident ray and reflecting it around the normal. The final color will be a combination of ordinary diffuse and glossy shading at the surface point and the color from *specular* (=mirror) reflection computed from the endpoint of the secondary ray (which might, in turn hit a reflective object and bounce further!).

3. For surfaces with transparency: an additional secondary ray is transmitted into the material (and possibly refracted).

   The final color is a combination of the diffuse shading, the color accumulated by the refractive and reflective rays.

The traditional strategy for implementing a CPU raytracer would involve pointers, recursion and dynamically allocated memory, none of which are available in GLSL. Instead, the skeleton code implements a ray stack as an array (in lieu of a proper function call stack for recursion) and a `push` operation.

Each time the ray hits a (partially) diffuse surface, the contribution `this_color` of this particular surface will be computed. The color for all rays (all `this_colors`) are summed into the final `color` that is returned by the `raytrace` function.

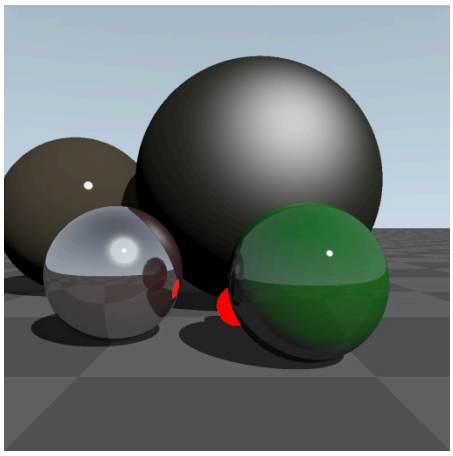
The `Ray` struct in this task also includes a `weight` element. The weight should be set to 1 initially, and will be decreased at each step along the path.

Tasks

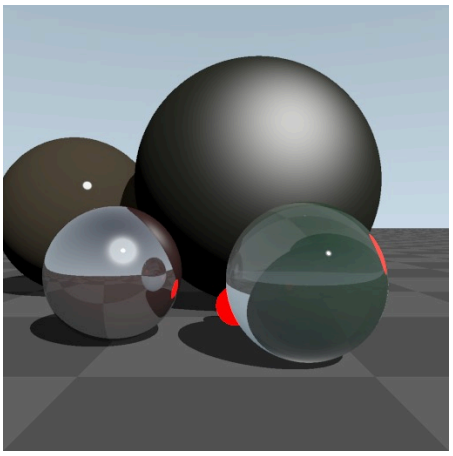
1. For all intersections, cast a shadow feeler ray towards the light source and check if it intersects an object on the way there. If so, set the irradiance to a small "ambient" value, and otherwise, use the irradiance from the light source as in the previous task. Remember to offset the start position of the shadow feeler with a small distance along its direction, to avoid "shadow acne".

   The "shadow feeler" should *not* be pushed on the ray stack, as there will be no recursive update from this direction.

   On the other hand, for objects with *specular* properties (i e, reflection or transmission) you need to push a new ray to the ray stack, for further processing.



(a) Ray tracing with shadows and specular reflection.



(b) Both transparency and reflection enabled for green sphere.

Figure 4: The scene with ray tracing instead of ray casting, from Task 4.2. Note that we now have objects casting shadows, as well as specular reflection.

2. For objects with reflective properties, shoot an additional ray, reflected about the normal (hint: use the `vec3 reflect(vec3 I, vec3 N )` function). Attribute a weight to the reflected ray (based on the amount of reflection of the material). The reflected ray will be handled in the next iteration of the loop in the `raytrace` function. Note that the new ray should get a portion of the weight assigned to the incoming ray; and the weight of the incoming ray must be reduced with the same amount, so that the total weight of all rays is equal to one. Remember to offset the start position of the reflected ray with a small distance along the reflection direction.

   Your result should now look like Figure 4a, with mirror reflections and shadows on the ground. However, don't worry too much if your scene doesn't look exactly like this example. The exact appearance depends on how the BRDF is implemented and how the material parameters are selected. But the *position* of the shadows and the reflections should be as in Figure 4.

3. For transmissive objects, it is slightly more complicated.

   The direction in which a transparency ray should be cast can be determined using the material's index of refraction as well as the *current* index of refraction (IOR). To do this properly, the index of refraction needs to be propagated through all calls to the raytracing function. However, to simplify things, you can assume that you don't have nested refractive objects, and that the IOR is always 1.0 (vacuum) on one side of the surface.

   You still need to check if the current ray is hitting a surface from the *outside* or the *inside* of the object. (Compare the surface normal and the ray direction to find out which side the ray is on. If the two vectors are pointing in the same direction, their dot product will be positive.) Compute the direction of the refracted ray using the IOR on the inside of the surface (`isec.material.ior`) and the IOR on the outside (1.0). The easiest way to compute the direction of the transmitted (refracted) ray is

to use the GLSL `refract` function. The third argument to this function is the ratio (IOR in)/(IOR out), so if the ray goes from air to glass, the ratio should be, e.g., 1.0/1.5.

If the ray is coming from the inside, the normal that you have in the `Intersect` struct will also be wrong, since that normal is pointing outwards.

Remember to offset the start position of the refracted ray with a small distance along the transmission direction; and shift part of the weight of the incoming ray to the new refracted ray.

The result should look like Figure 4b.

To be physically correct, you should compute the amounts of light being refracted into the material and reflected off the surface using the Fresnel Equation (or with Schlick's approximation of it).

Optionally, you may also include absorption by accounting for the length of the path the ray traversed inside the object and add some of its diffuse color (or a simple constant term, for simplicity). This should reduce the weight of the ray that comes out on the other side.

Questions

Question 3 How does the maximum depth (`MAX_DEPTH`) influence the appearance of the scene? How would a refractive object appear at a ray depth of 1, 2 and 3? How about reflective objects? At which depth does your scene no longer change by increasing the ray depth further? Why?

+1 point

4.3 Adding triangles

Now you should also add another type of primitive object: triangles. Since the ray tracer is implemented within the fragment shader, you can no longer use OpenGL's triangle handling, but have to do it yourself.

Tasks

1. Add a new `struct Triangle` to represent triangles.
2. In `init`, add a number of triangles, in addition to the objects that are already there. The intersection test may be rather heavy, so you probably don't want to have too many, but start with one, and maybe add a tetrahedron, as in Figure 5.
3. Add `a float intersect(Ray, Triangle)` function to handle the intersection test with triangles.
4. Include the triangle intersection testing in the main `intersect` function.

+2 points

4.4 Path tracing

Raytracing plausibly renders sharp shadows, reflection, and transparency, but still does not reproduce several phenomena that we expect to see in reality. For example, the shadow boundaries are sharper than we would expect from a light-source of non-infinitesimal size, the light is not transmitted and focused through the glass sphere (caustics), and there is no secondary illumination from one object onto another.

In this task, you will implement a path tracer in GLSL.

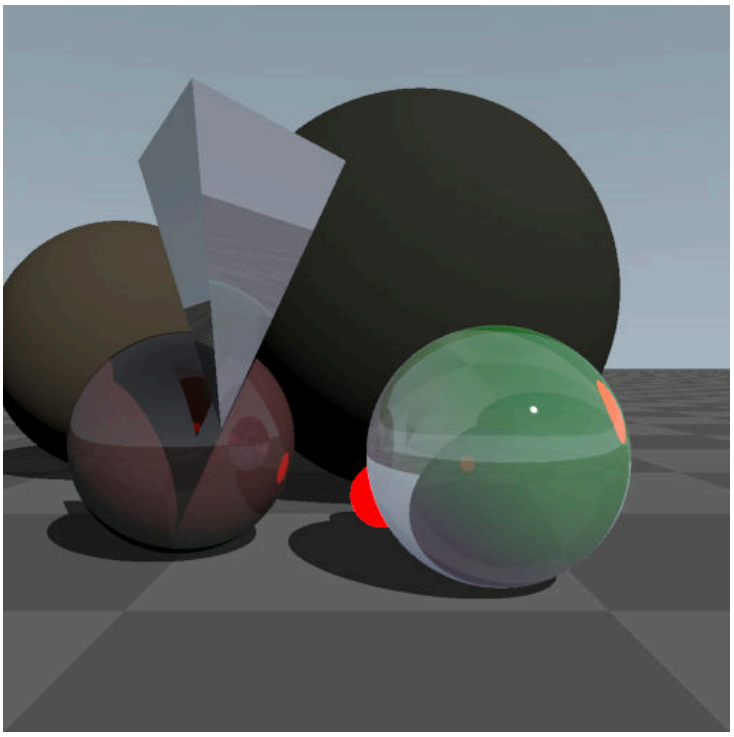


Figure 5: Ray traced triangles from Task 4.3.

Tasks

1. Begin by copying your fragment shader from the previous task, and replace the `vec3 raytrace()` function with `vec3 pathtrace(Ray ray)`. Instead of iterating over a ray stack, make a for-loop in `pathtrace` that iterates `MAX_DEPTH` times. This loop will trace a path up to `MAX_DEPTH` bounces. In turn, `pathtrace` should be called `MAX_SAMPLES` times from `main`, in order to trace so many paths per draw call, each with a random trajectory. For that purpose, also define a constant; e.g., `#define MAX_SAMPLES 20`. (This is still all done within the fragment shader.) From the main function in your fragment shader, shoot `MAX_SAMPLES` number of rays into your scene, and set the final frag color to the average result.

   You may want to remove the triangles from the scene now, to speed things up a little.

2. In `pathtrace`, add a `vec3` variable `coefficient` that keeps track of the color that gets transmitted (and not absorbed) along the path that is traced. The reasoning behind this is that on each particular path from the sensor to an emissive surface ("luminaire", or light source), the light may bounce on several objects of different colours. This will cause certain wavelengths to be absorbed until the path terminates at a light source. The `coefficient` should be $(1,1,1)$ to begin with. At each intersection, you should multiply `coefficient` the colour of the surface.

   As an example, imagine a green surface illuminated by a red light. Here a ray from the camera will hit the green surface and multiply the color green into its coefficient vector. Suppose the next bounce ends up at the red emitter. Since this is an emissive surface, we can add the emission

(red) times the colour transmitted along the path (green) resulting in a vector of all zeros. So the red light has no contribution of the colour of this pixel. (You could also check if the "coefficient" is close to zero and quit early by this approach, since this path will not generate any new contribution, even when hitting a white light source).

In other words, initialise

```
    vec3 coefficient = vec3(1);
    vec3 color = vec3(0);
```

3. Inside the for loop in `pathtrace`, first compute the intersection for the current path segment (using the raycast function from before).

4. If the ray hits an emissive surface, add `emission * coefficient` to the colour of the path.

   Note that the sky sphere in the example code emits light, so the whole sky sphere is now treated as a light source.

5. *Optionally*, implement `estimateDirectLightFromPointLights`. This can be taken from the raytracing code in the previous task, adding to `color` a sample from the reflectance equation; i e, light × BRDF × cosine for all light sources that are visible from the current intersection point. In the example scene, this is not necessary, since the whole sky is treated as a luminaire, but you can also add a point light source at the sun, which will give you more explicit highlights.

6. Now it is time to estimate the *indirect* light that does not come directly from a point light source. This includes both specular reflection and transmission, as well as diffuse/glossy scattering.

   Trace a new path segment (i e, a ray). The direction of the new path segment is determined randomly (as with the `scatter` function in the lecture slides). You should implement `scatter` by combining a broad distribution with specular paths (if the material includes specular components in its material definition).

   First, generate a uniform random number r between 0 and 1. There is no default `rand` function in GLSL, but a common way to implement random number generation is the following snippet

```
float seed = 0.;
float rand(void){return fract(sin(seed+=0.14) * 43758.5453123);}
```

   which you can add to the top of your fragment shader. This function returns a uniform random number between 0 and 1. Inside the `main` loop, to get a unique random seed per pixel, you can use

```
seed = i_global_time + i_window_size.y * gl_FragCoord.x / ↩
    i_window_size.x + gl_FragCoord.y / i_window_size.y;
```

   With probability *reflection amount*, the new ray should be traced in the mirror direction — that is, if r is less than than the reflection amount of the material at this intersection.

   With probability *transmission amount*, it should be traced in the refraction direction.

   And with probability (1 − reflection − transmission), it should be scattered "diffusely" (in a more or less uniformly random direction).

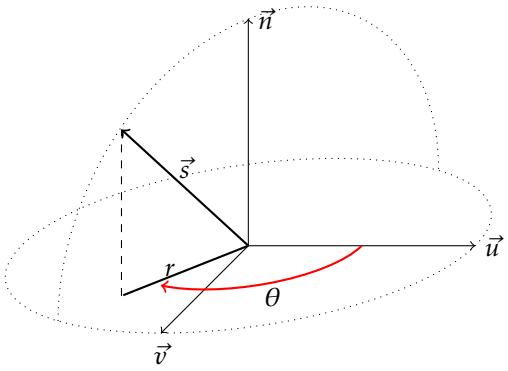


Figure 6: By projecting random points from a circular disc up to the hemisphere we get a distribution of samples that corresponds to weighing a uniform distribution with the cosine of the angle between the ray and the surface normal. Here, $\{\theta \mid \theta \in \mathbb{R} \text{ and } 0 \leq \theta \leq 2\pi\}$ and $\{r \mid r \in \mathbb{R} \text{ and } 0 \leq r \leq 1\}$ are uniform random variables. The vector $\vec{n}$ is the surface normal, orthogonal to both $\vec{u}$ and $\vec{v}$. The vector $\vec{s}$ is then a sampling direction from the appropriate probability distribution that is proportional to the cosine term in the rendering equation.

For this new path segment, you can just update the origin and direction of the ray parameter, and the next iteration of the for loop will handle the next segment. There is no need to create a new `Ray` object.

7. In the diffuse case, one can choose a number of different functions to sample from. Depending on the situation, it is sometimes good to sample from a function that is proportional to the shape of the BRDF, sometimes better to take more samples near the direction to the light source, etc (as described in Lecture 9). However, a "baseline" method is to sample proportional to the cosine to the normal vector, since light that comes along the normal direction is usually more important than light that comes at a very shallow angle.

   The following function returns a random direction within some angular region `spread` of a mean direction. Setting `mean` to the normal direction and `spread` to 1.0 returns a random direction on the positive hemisphere. (Note that you can also use this function in the specular cases, with a smaller spread, to get blurry reflections and transmissions.)

```
vec3 random_direction(vec3 mean, float spread)
{
  float r2 = spread*rand();
  float phi = 2.0*PI*rand();
  float sina = sqrt(r2);
  float cosa = sqrt(1. - r2);

  vec3 w = normalize(mean), u = normalize(cross(w.yzx, w)), ↩
      v = cross(w, u);
  return (u*cos(phi) + v*sin(phi)) * sina + w * cosa;
}
```

   See Figure 6 for an explanation of what this function does. It is a kind of *importance sampling*, proportional to the cosine of the direction.

8. The BRDF of the material should influence the `coefficient` here too.

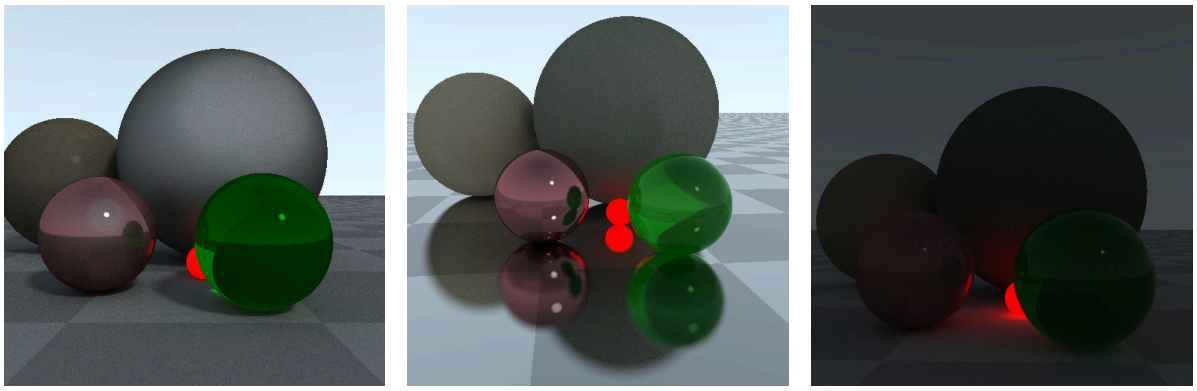


Figure 7: Path traced versions of the scene in Task 4.4. *Left:* with Blinn-Phong BRDFs. *Middle:* with Lambertian spheres, plus additional blurred reflections. *Right:* dimming down the sky (and cranking up the emission of the red ball somewhat).

Factor in the reflectance you get with the $\vec{\omega}_\mathrm{i}$ direction that you just sampled and the $\vec{\omega}_\mathrm{o}$ direction that goes back along your path.

Each path adds up the contribution of all emitters (and direct point lights) encountered along its way. The pathtrace function ultimately returns the sum of all paths that were traced, which you then need to normalize, by dividing by the number of samples. This is already implemented when setting `o_fragment_color` in the fragment shader code that you are given.

The result should look something like in Figure 7. Note the soft, but noisy, shadows around the spheres, the light coming from the emissive red ball, and the green caustic behind the green sphere.

Also try out the `random_direction` function in the specular cases, with a smaller spread, and try to get blurry reflection and transmission.

The skeleton code also has code to change the brightness of the sky using the mouse wheel. Now that we're no longer using a single point light source, you can try changing the brightness and observe what happens. Try dimming the sky until the only light is what is coming from the glowing red ball.

+1 point

4.5 Adding boxes

Add a `struct Box` object type and a ray/box intersection test (in a similar way as for the triangles in Task 4.3). You can implement the ray/box test on page 960 in the book (Real-Time Rendering, 4th edition), preferably with the optimisations for axis-aligned boxes (AABB). Implement a little Mincrafty grid world like in Figure 8.

Note that it is *not* necessary to do this with path tracing. You can complete this task with only ray casting, if you wish.

Returning the normal from the intersection test deserves a bit more explanation. One way is to do it in a similar way as the cube-map look-up that was shown in Lecture 5. Compute a vector $\vec{v}$ from the centre of the box to the intersection point that you have found (and normalise it). Compute $\vec{w} = \vec{v}/(\max(|v_x|, |v_y|, |v_z|))$. Then, one of the elements of $\vec{w}$ will be ± 1. Check which one, and set the normal accordingly. (E g, if $w_x < -0.99$, then the normal should be $(-1, 0, 0)$.)

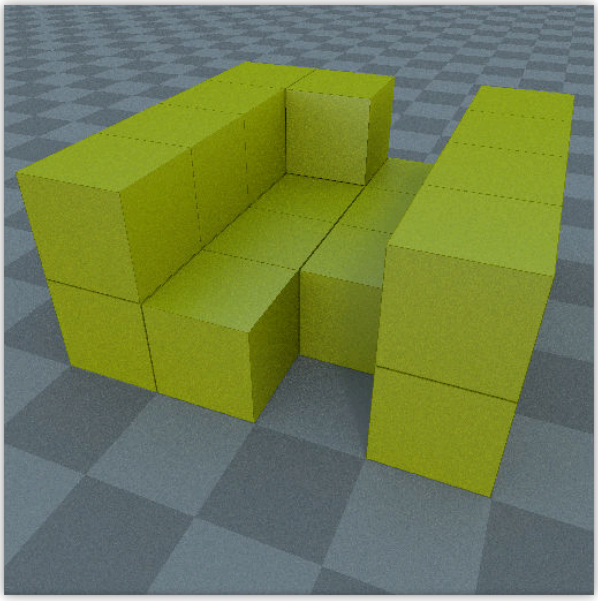


Figure 8: Boxes from Task 4.5